A Vision-Based Hand-Gesture Interface for Controlling a Simulated Robot

Author: Arjan Hayre. CID: 02137475. Repository: https://github.com/The-Asgardian/HCR

Abstract

This project builds a system that lets an inexperienced user drive a simulated mobile robot through a maze using only hand gestures seen by a standard webcam, with no special hardware. Hand landmarks are found with Google MediaPipe, turned into a direction and a speed level, and sent to the mecanum wheels of a KUKA youBot in the Webots simulator. In a short user study with three people who had never used the system, mean completion time across the three fell from 32.77 s on the first attempt to 27.59 s at their best, and everyone described the controls as easy to use.

1. Introduction

Controlling a robot usually needs special input hardware and some training. The aim here is a natural interface where the user points a finger in the direction they want the robot to go, and extends more fingers to go faster. The task is to drive through a maze from a start corner to a goal marker. To avoid needing physical hardware, the robot is simulated. The result is a full gesture pipeline, from webcam to robot, together with a small study of how quickly untrained users adapt to it.

2. Background

2.1 Robot simulators

Five common simulators were compared:

Simulator	Strengths	Weaknesses
Webots	Free and open source, one-click Windows install, built-in robots and sensors, in-process Python API, good docs, runs on CPU	ODE physics less exact than MuJoCo
CoppeliaSim	Many features, several physics engines, remote API	Heavier, steeper to learn
MuJoCo	Fast, accurate contact physics	Not ready to use out of the box; models and sensors are hand-built
Isaac Sim	Photorealistic, GPU sensors	Needs an RTX GPU, large install
Gazebo	Standard in the ROS world	Linux first, hard on Windows, tied to ROS

Webots (R2025a) was chosen. It installs in one click on Windows, runs on the CPU, ships the KUKA youBot with an accurate mecanum-wheel model, and runs Python controllers in-process for a fast edit and test loop.

2.2 Hand-gesture perception

Two toolkits were considered. Classic OpenCV methods (skin-colour segmentation, contours, convexity defects) are fragile under changing lighting and background. MediaPipe Hands gives a pretrained detector

that returns 21 hand landmarks in real time on the CPU, copes well with lighting and background, and needs no training data. The system uses MediaPipe for the landmarks and gesture logic, and OpenCV for camera capture, the on-screen overlay and video. The current MediaPipe Tasks API (HandLandmarker) is used, since the older solutions module has been removed in recent releases.

3. System Design

3.1 Pipeline

```
webcam -> MediaPipe HandLandmarker -> gesture classifier -> (direction, speed)
       -> mecanum inverse kinematics -> youBot wheels
```

The vision code runs inside the Webots robot controller. To keep the 16 ms timestep simulation responsive, detection runs at about 15 Hz while motor commands update every step.

3.2 How the control scheme developed

The interface was built up in stages, from a simple robot to a more capable one.

The first version used the differential-drive TurtleBot 3, the simplest mobile robot in Webots. Pointing a finger chose a compass direction, and the robot turned to face that direction and then drove forward. This worked, but it felt indirect, because a differential-drive robot cannot move sideways, so reaching any direction always meant turning first.

The second version replaced the robot with the KUKA youBot, which has four mecanum wheels and can move in any direction without turning. Pointing now moves the robot straight away in that world direction, with no turn. Speed control was then added by counting extended fingers, so the same pointing gesture also sets how fast the robot travels. This is the version used in the study.

3.3 Robot and control model

The youBot is holonomic, meaning it can move in any direction on the floor without changing which way it faces. The controller always commands zero rotation, so the body stays lined up with the world and a pointed compass direction maps straight to a movement. With no rotation, the wheel speeds follow the standard mecanum relation:

```
w1 = (vx + vy) / r
w2 = (vx - vy) / r
w3 = (vx - vy) / r
w4 = (vx + vy) / r
```

where vx is the East speed, vy is the North speed, and r is the wheel radius.

3.4 Gesture mapping

The index-finger vector (fingertip relative to knuckle) picks the world direction from its main axis. The number of extended fingers sets the speed. Extension is measured by comparing each fingertip distance from the wrist against the matching knuckle, which does not depend on hand orientation and still works when the hand points sideways.

Gesture	Command
Point index up, right, down or left	Move North, East, South or West
1 to 4 extended fingers	Speed level 1 to 4 (0.14 m/s per level)

Gesture	Command
Fist or no hand	Stop

A top-down camera with North upward lines up the screen with the direction the user points, so the controls match the user's own view.

3.5 Reliability

Single-frame errors are removed with a short delay filter: a command has to appear in four detections in a row before it is accepted, which removes flicker without any noticeable lag. Completion time and path length are logged automatically from an on-board GPS when the robot reaches the goal.

4. Experiments and Results

Task: drive the 5 by 5 m maze from the bottom-left start to the top-right goal marker. Measure: completion time, logged automatically. Each person did several timed runs, and the developer did five.

4.1 Developer runs

Run	1	2	3	4	5
Time (s)	26.34	25.52	24.70	24.61	24.66

Mean 25.17 s, standard deviation 0.76 s. Times drop from the first run and level off near 24.6 s, which shows a small learning effect and steady control from a practised user.

4.2 User study

Three family members with no prior experience each did four runs.

Run	Brother (s)	Mother (s)	Sister (s)
1	28.11	41.27	28.94
2	26.32	36.14	25.18
3	25.07	34.52	24.44
4	24.78	33.89	24.09
Mean	26.07	36.46	25.66
Std dev	1.52	3.35	2.23

4.3 Summary

Participant	First run (s)	Best (s)	Improvement	Mean and std dev (s)
Developer	26.34	24.61	6.57%	25.17 +/- 0.76
Brother	28.11	24.78	11.85%	26.07 +/- 1.52
Mother	41.27	33.89	17.88%	36.46 +/- 3.35
Sister	28.94	24.09	16.76%	25.66 +/- 2.23

For all four people the first run was the slowest, and everyone was faster on their second attempt. Because the maze is simple, users found a good route straight away rather than needing many tries. The largest

gain came from the least experienced person (Mother, 7.38 s faster), whose first run was slowest but who settled to a steady time near 34 s. The other participants dropped to around 24 to 26 s, close to the developer's times. Standard deviation falls as people get used to the controls, from 3.35 s down toward 0.76 s, which shows steadier control with practice.

When asked, all three said the controls were easy to use and made sense after a one-sentence explanation. Their main suggestion was that they would prefer a steering-wheel style controller, with smooth continuous steering rather than four fixed directions.

5. Discussion and Improvements

- Continuous steering-wheel control. The most requested change. Map hand tilt to a smooth heading and hand height to speed, giving continuous control instead of four fixed directions.
- Heading drift. Holonomic control assumes the body stays lined up with the world, but wheel slip can cause a slow drift over long runs. Adding a compass and correcting the heading would remove it.
- Gesture reliability. Detection depends on lighting and on keeping the hand roughly upright. A confidence threshold and a smoothing filter on the landmarks would steady the input further.
- Autonomy. An optional gesture to switch on lidar-based obstacle avoidance would add a safety layer for a new user.

6. Conclusion

A full webcam-to-robot gesture interface was built and tested. MediaPipe landmarks drive a holonomic youBot through a maze in Webots, using pointing for direction and finger count for speed. The scheme grew from a simple differential-drive robot that turned to face a direction, to an omnidirectional robot that moves in any direction at once with finger-controlled speed. Untrained users learned it within a single run and reached near-developer times quickly, which shows the interface is easy for inexperienced operators. The clearest next step is continuous steering-wheel style control, which users found more natural than fixed directions.

References

1. O. Michel, "Webots: Professional Mobile Robot Simulation," Int. J. of Advanced Robotic Systems, 1(1), 2004.
2. C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.
3. F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv:2006.10214, 2020.
4. G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
5. E. Rohmer, S. P. N. Singh, M. Freese, "V-REP: A versatile and scalable robot simulation framework," IEEE/RSJ IROS, 2013.
6. E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," IEEE/RSJ IROS, 2012.
7. G. Casiez, N. Roussel, D. Vogel, "One Euro Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems," ACM CHI, 2012.
8. KUKA youBot, Webots documentation, Cyberbotics Ltd.